Automated Data Health & Profiling Dashboard

Phase 1 & 2 CLI Tool - Work Completed (Plain-Language Guide)

Updated 2026-09-26. Written so it can be understood without prior knowledge of this project.

1. What This Project Does, In Plain Language

Picture a spreadsheet full of raw data - rows of records, columns of numbers and text, and inevitably some blank cells and messy entries. This tool is a health check-up for that spreadsheet. You hand it a CSV file (a plain-text spreadsheet format) and it does two things, one after the other:

- **Describes your data** - how many rows and columns, what type each column is, how many blanks it has, and standard statistics (average, spread, etc.) for every numeric column.
- **Flags unusual rows** - using two machine learning techniques, Isolation Forest and KNN, it looks across all numeric columns at once and points out rows that look out of place compared to the rest of the data, then compares what the two techniques found.

Everything currently happens in a terminal window - you type a command, and the results print as text tables. There is no visual app yet; that comes in a later phase (see the separate Remaining Work document).

2. The Tools We Used, Explained

None of this was built from scratch - it leans on well-established, free, open-source building blocks. Here is what each one is and why it's in this project:

Tool	What it is, in plain terms
Python	The programming language everything is written in - the set of instructions the computer follows.
CSV file	"Comma-Separated Values" - a plain-text version of a spreadsheet. Each line is a row; commas separate the columns. Excel, Google Sheets, and databases can all export to this format.
pandas	The library that loads a CSV into a table-like structure in memory, called a DataFrame. It's what actually counts rows, finds blanks, and computes averages - basically Excel's formula engine, but in code.
NumPy	A fast, low-level number-crunching library. pandas and scikit-learn are both built on top of it; you don't call it directly, but it's doing the arithmetic underneath.
scikit-learn	A widely-used machine learning library. It provides ready-made, tested pieces so nobody has to write them from scratch: the Isolation Forest algorithm, the nearest-neighbour search behind KNN, and the tool that puts columns on the same scale.
tabulate	Turns a plain table of numbers into a nicely bordered text table for the terminal (the boxed-in tables you see in the output). If it isn't installed, the program still works - it just prints a plainer table instead.

argparse	Built into Python itself. It reads whatever you typed after "python cli.py" on the command line - the file path, and options like --method, --contamination and --k - and hands them to the program as normal variables.

3. Files Created

File	Purpose
cli.py	Entry point: reads what you typed, calls everything else, prints the results
stats.py	Calculates the Phase 1 descriptive statistics
ml_processing.py	Runs Isolation Forest and KNN to find unusual rows, and compares them
preprocessing.py	Fills in missing numeric values before the two methods run
requirements.txt	The list of tools above, so they can all be installed in one step
sample_numeric.csv / sample_messy.csv / sample_fully_missing_col.csv / sample_200.csv	Practice CSV files used to test the program

4. The Statistics We Calculate, Explained Simply

For every numeric column (a column that's actually made of numbers, not text), the program calculates a small set of standard statistics. Here's what each one means, in plain terms:

Mean, Median, and Mode - three ways to describe a "typical" value

- **Mean** (average) - add up every value, divide by how many there are. Easily skewed by one huge or tiny number.
- **Median** - sort all the values and pick the middle one. Much less affected by extreme values than the mean - useful when data has outliers.
- **Mode** - the value that shows up most often.

Variance and Standard Deviation - how spread out the values are

Variance measures, on average, how far each value sits from the mean (squared, so bigger gaps count much more). **Standard deviation** is just the square root of variance, which brings it back into the original units (e.g. dollars, years) so it's easier to read as "the typical distance from average."

Kurtosis - how "extreme" the extremes are

Kurtosis describes the shape of the tails of your data's distribution. A normal, bell-curve-shaped column has a kurtosis near zero. A high kurtosis means the column has heavier tails than normal - i.e. more/bigger extreme values than you'd expect - which is often an early hint that outliers are present, before you even run the outlier-detection step.

Five-Point Summary - a quick snapshot of the range

Min, Q1 (25% of values fall below this), Median (the 50% mark), Q3 (75% of values fall below this), and Max. This is the same set of five numbers a box plot draws - it tells you the full range and where the "middle chunk" of your data sits.

Covariance Matrix - do columns move together?

For every pair of numeric columns, covariance tells you whether they tend to rise and fall together. A positive number means "as one goes up, the other tends to go up too" (e.g. age and income, often). A negative number means they move in opposite directions. A number close to zero means they don't move together in any obvious way. The matrix just shows this for every pair of columns at once, including each column against itself (which is just its variance).

5. The Models: Isolation Forest and KNN

"Outlier detection" or "anomaly detection" just means: find the rows that don't look like the rest. A simple approach would be to flag anything more than, say, 3 standard deviations from the mean of one column. The limitation is that it only looks at one column at a time - it can miss a row that isn't extreme in any single column, but is still an unusual *combination* (e.g. a very young person with a very high income - neither number alone is shocking, but together they're unusual). Both methods below look at all the numeric columns at once, which catches those combination cases too.

Model 1 - Isolation Forest: how easily can this row be cut off?

Think of a game of 20 Questions played on your data instead of a person. The algorithm repeatedly picks a random column and a random cut-off value, and splits the data in two: "is this row's value above or below the cut-off?" It keeps splitting randomly, over and over, building a tree of yes/no questions, until every row is on its own.

A row that's **similar to many other rows** is surrounded by close neighbors, so it takes many random splits to separate it from everyone else - it's "hard to isolate." A row that's **unusual** stands apart from the crowd, so a handful of random splits is often enough to isolate it on its own - it's "easy to isolate." The algorithm repeats this random-splitting process many times over (building a whole "forest" of these random trees, hence the name), and averages the results for reliability.

> **Analogy:** imagine a crowded room where most people are standing in one big cluster chatting, and one person is standing alone in a far corner. If you kept drawing random lines across the room and asking "who's on this side?", you'd isolate the lone person in the corner within a line or two. Separating any one person out of the big cluster would take many more lines, because they're packed in with everyone else.

Two settings you'll see in the output

- **Anomaly score** - roughly, how few splits it took to isolate that row, averaged across the whole forest. A more negative score means the row was easier to isolate, i.e. more unusual. The flagged rows are sorted by this, most unusual first.
- **Contamination** - a setting you choose (default 5%, changeable with --contamination) that tells the model roughly what fraction of your data you expect to be unusual. The model uses it to decide where to draw the line between "normal" and "flagged." Raising it flags more rows; lowering it flags fewer. Both models share this setting.

Model 2 - KNN (K-Nearest Neighbours): how far is this row from its neighbours?

KNN starts from a much simpler idea: a normal row has plenty of similar rows close to it, while an unusual row sits far away from everything else. For every row, the tool finds its 5 closest rows (its "nearest neighbours") and measures how far away the 5th one is. A big distance means the row is out on its own, so it gets a high **knn_distance** score - and the rows with the highest scores are flagged.

> **Analogy:** in the same crowded room, ask everyone "how far would you have to walk to reach your fifth-closest neighbour?" People inside the big group only take a step or two. The person alone in the corner has to cross the room.

- **Columns are put on the same scale first** ("standardised"). Distance only makes sense if age (tens) and income (tens of thousands) count equally - otherwise income would drown out everything else. Isolation Forest does not need this step.
- **The same contamination setting** picks how many rows are flagged (the top 5% of distances by default), so both methods flag the same number of rows and can be compared fairly.
- **k = 5 neighbours by default** - change it with --k.
- **No randomness** - the same file always gives exactly the same answer.

Why use both?

- They look for different things: Isolation Forest asks "how easily can this row be cut off from the others?"; KNN asks "how far is this row from the rows closest to it?" A row can look unusual to one and ordinary to the other.
- A row flagged by **both** methods is very likely a real outlier. A row flagged by only one deserves a closer look but is less certain.
- The tool prints both lists, then a comparison with the rows both methods agree on shown first.

6. Preprocessing: Filling In the Gaps (Median Imputation)

Real-world data almost always has blanks - someone skipped a field, a sensor didn't report, etc. Both methods (like most machine learning models) cannot work with a blank cell - every row they score needs a complete set of numbers. The original plan for this phase simply skipped any row with a blank; you later asked for the gaps to be filled instead so every row gets scored.

What "median imputation" means

For each numeric column, the program looks at all the values that *aren't* blank, finds that column's median (the true middle value), and uses it to fill in every blank cell in that column. The median is used instead of the mean because it isn't dragged around by extreme values - so a single huge outlier in a column doesn't distort the value used to fill its blanks.

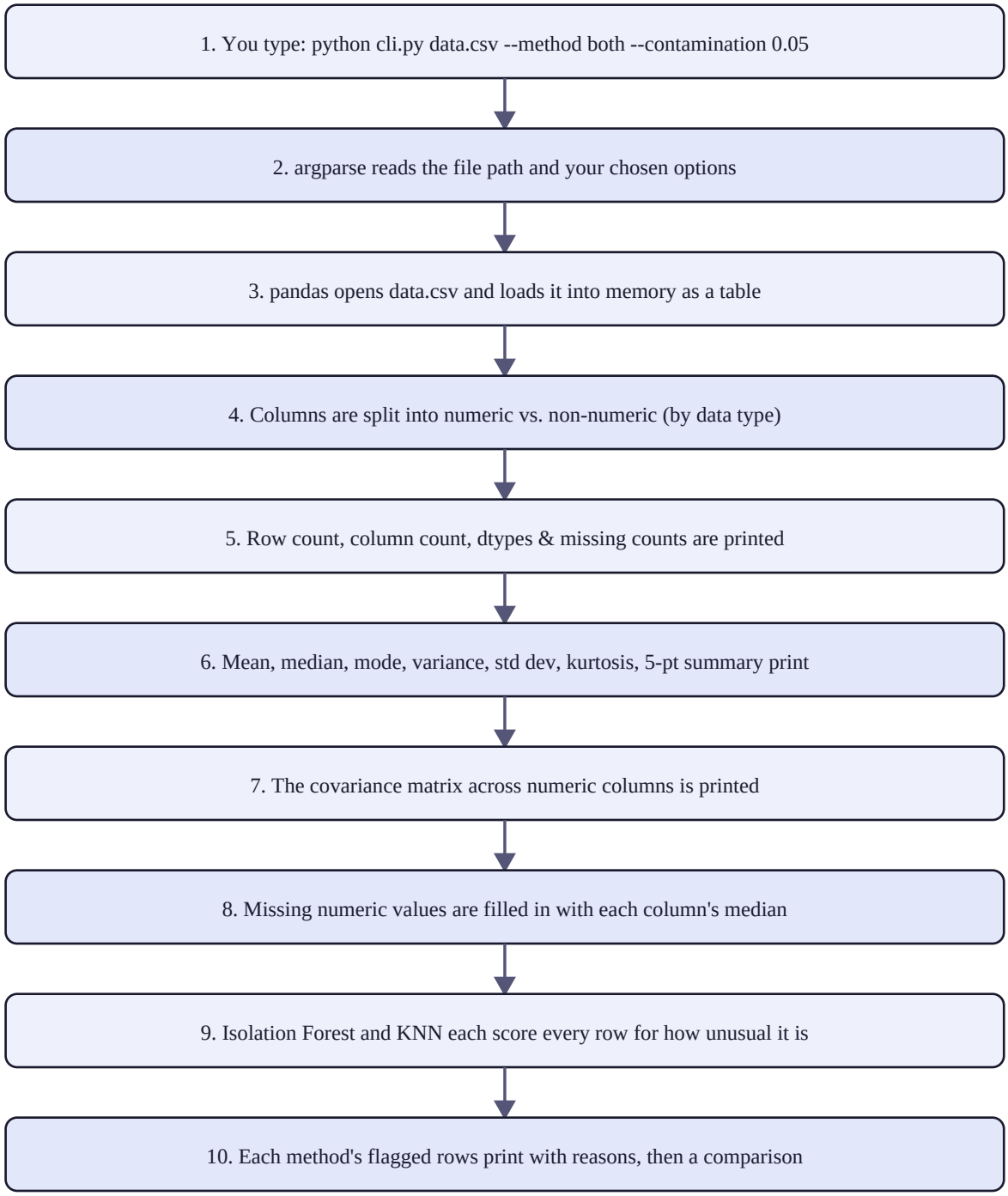
One edge case is handled explicitly: if an entire column is blank, there's no median to compute at all. Rather than crashing, the program excludes that column from the outlier check and says so plainly in the output.

Important: this filling-in only happens right before the two methods run. The descriptive statistics printed earlier (Section 4 above) are still calculated from the real, original data with its real gaps - so they keep

reflecting genuine data quality, not filled-in guesses. A known trade-off: if the blanks land on exactly the values that made a row unusual, filling them with the median can hide that row (see Section 10).

7. Detailed Flow: What Happens When You Run the Program

Here is the complete journey of your data, from the moment you type a command to the final line printed in the terminal:



Walking through each step

- **Steps 1-2 - reading your command.** Nothing happens to your data yet; the program is just figuring out which file to open and which options you chose (or falling back to the defaults: both methods, contamination 0.05, k = 5, imputation on). Impossible settings, like a contamination of 0.9, get a short usage message instead of a crash.
- **Step 3 - loading the file.** If the file path is wrong, empty, or badly broken, the program prints a clear one-line error and stops here on purpose, instead of crashing with a wall of technical text.
- **Step 4 - sorting columns.** pandas already knows, from reading the file, whether a column looks like numbers or like text. Any column that isn't cleanly numeric (even one stray text value in it) is treated as non-numeric and excluded from the statistics and outlier detection - this is why a "messy" column with even one bad entry won't crash the numeric calculations.
- **Steps 5-7 - describing the data.** These are pure description: no data is changed, nothing is guessed or filled in. This is the true, as-is picture of your file.
- **Step 8 - filling gaps.** Only now, right before the machine learning step, are blanks filled in (median per column) - and only in a working copy used for scoring, not in the statistics you already saw.
- **Step 9 - finding outliers.** The now-complete numeric data is handed to Isolation Forest, and a standardised copy is handed to KNN. Each scores every row.
- **Step 10 - reporting results.** For each method the program shows how many rows were scored, how many had gaps filled, and the flagged rows with a plain-language reason. Then it compares the two: how many rows both flagged, how well their scores agree overall, and a single table with the rows both methods agree on at the top. Nothing is written to a file; it's all terminal output.

8. How to Run

```
python -m pip install -r requirements.txt
python cli.py path/to/data.csv
python cli.py path/to/data.csv --method knn --k 7
python cli.py path/to/data.csv --method isolation_forest
python cli.py path/to/data.csv --contamination 0.1
python cli.py path/to/data.csv --no-impute
```

--method chooses isolation_forest, knn, or both (the default). --k sets how many neighbours KNN looks at. --contamination sets the share of rows to flag for both methods. --no-impute skips incomplete rows instead of filling them.

9. Testing Performed

- Messy CSV with mixed-type columns, blank cells, and a ragged row - ran with no crash; mixed-type columns correctly fell into the non-numeric bucket.
- Clean multi-column numeric CSV - verified full numeric statistics, five-point summary, and covariance matrix across several columns.
- Missing file path - produced a clean error message and exit code 1 instead of a traceback.
- CSV with a numeric column that is entirely missing - correctly excluded from scoring and reported, without crashing; other columns' missing values still imputed and scored.

- Edge cases for both methods: a file with no numeric columns, a single-row file, a file where every row has a gap combined with --no-impute, and a k larger than the number of rows - all handled with clear messages, none crash.
- Impossible settings (--contamination 0 or 0.9, --k 0, an unknown --method) give a short usage message instead of a crash.
- Isolation Forest's output on the test files was compared before and after KNN was added and stayed identical (apart from one reworded sentence).

10. How Well Did It Work? (Test With Hidden Answers)

A 200-row practice file was built with 8 unusual rows deliberately hidden inside it, and their identities written down separately. Both methods were then run on the actual CSV file, each allowed to flag 10 rows (5%).

Method	Rows flagged	Hidden rows found
Isolation Forest	10	7 of 8 (87.5%)
KNN	10	8 of 8 (100%)
Flagged by both methods	8 (12 different rows flagged in total)	7 hidden + 1 naturally extreme row

- Isolation Forest missed one hidden row (id 145). The practice file's random blanks happened to wipe out its income and score - the two values that made it stand out - and filling them with the median made it look ordinary. KNN still caught it through its age and tenure.
- If each method may flag 20 rows (contamination 0.10), both find all 8 - at the cost of more extra rows to look through.
- The two methods' scores agree well across all 200 rows (rank correlation 0.86), yet not perfectly - which is exactly why using both is worthwhile.
- Honest limits: this is a controlled test on made-up data with known answers. On a real file there is no answer key, so a flag can be checked for plausibility but not proven right or wrong.

11. Acceptance Checklist (from project_plan.md, plus additions)

Item	Status
python cli.py data.csv runs without errors on a real, messy CSV	Done
Dataset overview prints (rows, columns, dtypes, missing counts)	Done
Mean, median, mode, variance, std dev, kurtosis print per numeric column	Done
Five-point summary prints per numeric column	Done
Covariance matrix prints	Done
--contamination flag works and defaults sensibly	Done
Isolation Forest outlier count and flagged rows print to terminal	Done

Skipped/incomplete rows are called out in the output	Done
(Added) Missing numeric values imputed by default before scoring	Done - extra
(Added) Plain-language reason printed for every flagged row	Done - extra
(Added) KNN outlier detection (standardised features, --k)	Done - extra
(Added) Side-by-side comparison of Isolation Forest and KNN	Done - extra
(Added) Both methods checked against planted outliers with known answers	Done - extra